



Tecnológico de Monterrey

Campus Guadalajara

Escuela de Ingeniería y Ciencias

TC3002B

Desarrollo de Aplicaciones Computacionales Avanzadas

Módulo 3 — Compiladores

Equipo:

Gonzalo Calderon Gil Leyva - A01740008

Jorge Antonio Arizpe Cantú - A01637441

Jesús Enrique Bañales López - A01642425

Edwin Iñiguez Moncada - A01637064

Diego Alejandro Michel Castro - A01641907

1. Introducción

1.1 Resumen

El presente documento reporta el análisis, diseño, implementación y validación de un analizador léxico (scanner) para un subconjunto del lenguaje Triton GPU Kernel. Triton es un lenguaje específico de dominio (DSL) desarrollado por OpenAI, con sintaxis tipo Python, orientado a la programación de kernels para GPU. Dado que la sintaxis superficial de Triton es esencialmente la de Python, el subconjunto léxico que se reconoce en este proyecto corresponde al de Python; las primitivas propias de Triton (por ejemplo `tl.load`, `tl.store`, `program_id`) son tratadas como identificadores ordinarios del lenguaje, sin reglas léxicas adicionales.

El scanner está dividido en catorce módulos independientes, cada uno encargado de una familia de tokens. Esta separación responde a una decisión pedagógica: cada módulo implementa de forma explícita el autómata finito determinista (DFA) que reconoce los tokens de su familia, lo que permite leer el código en paralelo con la tabla de transición del diseño y verificar que ambas representaciones coinciden estado por estado.

El reporte se organiza siguiendo la estructura solicitada en la guía oficial: Sección 1 (Introducción) describe el modelo conceptual y la elección de la herramienta de implementación; Sección 2 (Análisis) presenta la especificación informal y formal de los tokens y errores; Sección 3 (Diseño) muestra los autómatas y las tablas de transición más eficientes para cada familia; Sección 4 (Implementación) explica el código módulo por módulo; Sección 5 (Verificación y Validación) presenta los casos de prueba con sus resultados esperados; Sección 6 (Plan de Trabajo) enumera las fases de desarrollo; y finalmente la Sección 7 lista las referencias bibliográficas en estilo IEEE.

1.2 Notación

1.2.1 Máquinas de estado finito y expresiones regulares

El modelo formal utilizado a lo largo del reporte es el autómata finito determinista (DFA, por sus siglas en inglés). Un DFA se define como una quintupla $M = (Q, \Sigma, \delta, q_0, F)$, donde Q es un conjunto finito de estados, Σ es el alfabeto de entrada, $\delta : Q \times \Sigma \rightarrow Q$ es la función de transición, $q_0 \in Q$ es el estado inicial y $F \subseteq Q$ es el conjunto de estados de aceptación. Cada estado de aceptación lleva asociado un identificador de token (Token ID), de modo que la llegada del autómata a ese estado durante la lectura del archivo fuente implica la emisión del token correspondiente.

Las expresiones regulares se utilizan en este reporte como una notación más compacta para describir las mismas familias de cadenas que reconocen los DFA; por el teorema de Kleene, ambas formalizaciones son equivalentes [1]. En este documento, las expresiones regulares describen la especificación formal de cada categoría de token, mientras que los DFA describen el algoritmo concreto de reconocimiento. Ambas representaciones se presentan en paralelo para que el lector pueda verificar su equivalencia por inspección.

Una tabla de transición es la representación tabular de la función δ . Sus filas se indexan por estados y sus columnas por símbolos del alfabeto (o por clases de equivalencia de símbolos cuando el alfabeto es grande). Cada celda contiene el estado siguiente, un marcador de aceptación o un marcador de error. La tabla de transición es lo que se consultaría mecánicamente para simular el autómata carácter por carácter.

1.2.2 Justificación del modelo de desarrollo

El proceso de desarrollo seguido (Análisis → Diseño → Implementación → Verificación y Validación) corresponde al modelo clásico de la ingeniería de software adaptado al dominio de los compiladores [1]. Este modelo es particularmente limpio para la fase léxica porque la salida de la fase de análisis (las expresiones regulares) puede traducirse mecánicamente a la fase de diseño (los DFA) y de ahí a la implementación (el código del scanner). Mantener visibles las cuatro fases del proceso en el reporte hace inmediata la trazabilidad requisito → diseño → código → prueba.

1.2.3 Justificación del lenguaje de implementación

La especificación original del proyecto sugiere el uso de la herramienta UNIX lex (o flex). El líder de la Unidad de Aprendizaje autorizó explícitamente la sustitución de lex por cualquier lenguaje de programación, siempre y cuando se preserve la esencia académica del análisis léxico. Se eligió Python 3 con autómatas implementados manualmente por las siguientes razones.

Primero, la prohibición de utilizar bibliotecas de expresiones regulares se respeta de forma estricta: ninguno de los catorce módulos importa el módulo `re` ni utiliza la API de expresiones regulares de Python. Segundo, la prohibición sobre el módulo `ast` también se respeta. Tercero, y lo más importante desde el punto de vista didáctico, los autómatas se escriben en código fuente como cadenas de `if/elif` sobre la variable `estado_actual`, lo cual hace que el modelo formal sea directamente legible en el código. Una implementación con lex compilaría las expresiones regulares a una tabla opaca en tiempo de construcción, ocultando el DFA al desarrollador; nuestra implementación lo expone.

En los módulos donde el comentario indica «Tipo de implementación: REGEX», la palabra `regex` se refiere a la especificación formal del token (la expresión regular que lo define), no al uso de la biblioteca `re`. La estrategia concreta de reconocimiento, en esos módulos, consiste en

comparar la cadena de entrada carácter por carácter contra la cadena esperada del lexema (por ejemplo, contra el literal "def" para la palabra reservada DEF), seguido de una verificación de que el siguiente carácter no extiende el identificador. Esto es una forma de DFA degenerada con un único camino lineal de estados, y es lo que se conoce como reconocedor por comparación directa.

2. Análisis

2.1 Requisitos funcionales del scanner

Los requisitos funcionales del scanner se derivan directamente de la especificación del proyecto (Sección II del documento de instrucciones), del archivo `Lexer.csv` proporcionado por el profesor con la lista oficial de tokens a reconocer, y del archivo `Tabla_Simbolos_Ejemplo.csv` que ilustra la estructura esperada de la tabla de símbolos. Los requisitos identificados son los siguientes:

1. RF1: Reconocer las dieciocho palabras reservadas del subconjunto aprobado, emitiendo un token con identificador propio para cada una.
2. RF2: Reconocer identificadores de usuario (NAME) siguiendo las reglas léxicas de Python: una letra o guion bajo seguido de letras, dígitos o guiones bajos.
3. RF3: Reconocer literales numéricos enteros y decimales (NUMBER), descartando los lexemas mal formados como un punto sin dígitos posteriores.
4. RF4: Reconocer literales de cadena (STRING) delimitados por comillas dobles, sin saltos de línea internos.
5. RF5: Reconocer todos los operadores aritméticos, incluyendo los compuestos por dos caracteres como `**` y `//`, aplicando la regla del maximal munch (se acepta el lexema válido más largo).
6. RF6: Reconocer todos los operadores relacionales, de asignación simple y de asignación compuesta del subconjunto.
7. RF7: Reconocer todos los delimitadores y símbolos especiales del subconjunto, incluyendo los compuestos por dos caracteres como `->`, `<<`, `>>`.
8. RF8: Manejar la indentación al estilo Python: emitir tokens `NEWLINE`, `INDENT` y `DEDENT` siguiendo el algoritmo de la pila de niveles.
9. RF9: Detectar y reportar los errores léxicos con mensajes claros, sin detener el análisis tras encontrar un error.
10. RF10: Construir la tabla de símbolos para los identificadores y literales que lo requieran, registrando también los lexemas inválidos como entradas de tipo «(desconocido)» según el ejemplo proporcionado.

2.2 Especificación informal del subconjunto léxico

El subconjunto léxico aprobado se compone de seis grandes categorías de tokens: palabras reservadas, identificadores y literales, operadores, delimitadores, símbolos especiales y tokens de control de presentación (indentación). A continuación se describe cada categoría en lenguaje natural; la formalización mediante expresiones regulares se presenta en la Sección 2.3.

Palabras reservadas (18 tokens)

Son cadenas fijas de letras que el lenguaje reserva con un significado predefinido y que no pueden usarse como identificadores: `def`, `return`, `if`, `else`, `elif`, `for`, `while`, `in`, `is`, `and`, `or`, `not`, `True`, `False`, `None`, `pass`, `break`, `continue`. La distinción entre mayúsculas y minúsculas se respeta estrictamente: `True` con T mayúscula es la keyword `TRUE`, mientras que `true` en minúsculas es un identificador `NAME`. Los nombres `triton`, `tl`, `jit`, `program_id`, `load`, `store`, `dot` y similares NO son palabras reservadas del lenguaje, sino identificadores ordinarios provistos por la biblioteca de Triton; el scanner los reconoce como `NAME`, lo cual respeta el principio de que el análisis léxico no debe depender de la semántica de biblioteca.

Identificadores y literales (3 tokens)

`NAME` corresponde a una secuencia de letras (mayúsculas o minúsculas), dígitos y guiones bajos, con la restricción de que el primer carácter debe ser una letra o un guion bajo. `NUMBER` corresponde a una secuencia de uno o más dígitos, opcionalmente seguida de un punto decimal y uno o más dígitos; los lexemas como `3.` (punto sin dígitos posteriores) son inválidos en este subconjunto. `STRING` corresponde a una secuencia arbitraria de caracteres delimitada por comillas dobles, sin saltos de línea internos.

Operadores aritméticos (7 tokens)

Son los operadores binarios estándar de Python: `+` (`PLUS`), `-` (`MINUS`), `*` (`TIMES`), `/` (`DIVIDE`), `//` (`FLOORDIV`), `%` (`MOD`) y `**` (`POWER`). Los operadores compuestos `**` y `//` comparten el primer carácter con sus contrapartes simples `*` y `/`, lo cual obliga al scanner a leer el siguiente carácter antes de decidir qué token emitir. Esta técnica se denomina lookahead de un carácter.

Operadores relacionales y de asignación (11 tokens)

Comparación: `<` (`LT`), `>` (`GT`), `<=` (`LE`), `>=` (`GE`), `==` (`EQ`), `!=` (`NE`). Asignación simple: `=` (`ASSIGN`). Asignación compuesta: `+=` (`PLUSEQ`), `-=` (`MINUSEQ`), `*=` (`TIMESEQ`), `/=` (`DIVEQ`). Aquí también se aplican el lookahead y el maximal munch: el carácter `'='` suelto es `ASSIGN`, pero seguido de otro `'='` se convierte en `EQ`; el carácter `'+'` suelto es `PLUS`, pero seguido de `'='` se convierte en `PLUSEQ`.

Delimitadores y símbolos especiales (17 tokens)

Delimitadores de un solo carácter: `( ) [ ] { } , : . @ ~ & | ^`. Símbolos especiales y delimitadores de dos caracteres: `->` (`ARROW`, usado para anotaciones de tipo de retorno en definiciones de función), `<<` (`LSHIFT`) y `>>` (`RSHIFT`). El módulo `SPECIAL_SYMBOLS.py` centraliza el reconocimiento de todos estos lexemas mediante una función auxiliar compartida, `comparar_símbolo`, que recibe el símbolo esperado y el nombre del token.

Tokens de control de presentación (3 tokens)

Python utiliza la indentación para delimitar bloques. Por ello, el scanner debe emitir tres tokens adicionales que en otros lenguajes no existirían: NEWLINE marca el final de una línea lógica, INDENT marca el aumento del nivel de indentación al comenzar una nueva línea con más espacios al inicio que la anterior, y DEDENT marca la disminución del nivel. La emisión correcta de INDENT y DEDENT requiere mantener una pila de niveles vistos previamente; este algoritmo se describe en detalle en la Sección 3.6.

2.3 Especificación formal mediante expresiones regulares

La siguiente tabla recoge la expresión regular que define cada categoría de token. La notación usada es la estándar: la barra | denota alternación, el asterisco denota cero o más repeticiones, el signo + denota una o más repeticiones, los corchetes denotan clases de caracteres, y la concatenación es implícita. Las cadenas literales se muestran entre comillas simples para distinguirlas de los metacaracteres.

Categoría	Token	Expresión regular formal	Ejemplo válido
Identificador	NAME	(letra '_') (letra dígito '_')*	x_ptr, BLOCK_SIZE
Literal	NUMBER	dígito+ ('.' dígito+)?	42, 3.14
Literal	STRING	""" (carácter sin """ ni '\n')* """	"hola"
Keyword	DEF	'd' 'e' 'f' (luego no-alfanum)	def
Keyword	RETURN	'r' 'e' 't' 'u' 'r' 'n'	return
Keyword	IF	'i' 'f'	if
Keyword	ELIF	'e' 'l' 'i' 'f'	elif
Keyword	ELSE	'e' 'l' 's' 'e'	else
Keyword	WHILE	'w' 'h' 'i' 'l' 'e'	while
Aritmético	PLUS	'+'	+
Aritmético	TIMES	'*' (no seguido de '*' ni '=')	*
Aritmético	POWER	'*' '*'	**
Aritmético	FLOORDIV	'/' '/'	//
Relacional	EQ	'=' '='	==
Relacional	NE	'!' '='	!=
Relacional	LE	'<' '='	<=
Asignación	ASSIGN	'=' (no seguido de '=')	=
Asignación	PLUSEQ	'+' '='	+=
Delimitador	LPAREN	'('	(
Delimitador	ARROW	'.' '>'	->

Categoría	Token	Expresión regular formal	Ejemplo válido
Delimitador	LSHIFT	'<' '<'	<<

Tabla 1: Especificación formal mediante expresiones regulares (selección).

2.4 Justificación de los tokens incluidos

Cada token incluido en el subconjunto corresponde a una construcción sintáctica que un kernel Triton típico utiliza: definición de función (def, return, ->), control de flujo (if, else, elif, for, while, break, continue, pass), expresiones lógicas y de bits (and, or, not, &, |, ^, ~, <<, >>), expresiones aritméticas y relacionales, acceso a atributos (DOT), llamadas a función (paréntesis, comas), suscripción (corchetes) y decoración (@). Esta cobertura permite analizar léxicamente programas Triton reales como el kernel de suma vectorial mostrado en la Sección 5.

Las palabras reservadas import, from, as y class fueron deliberadamente excluidas del subconjunto porque no aparecen en el archivo Lexer.csv proporcionado por el profesor. Si el subconjunto necesitara extenderse para cubrir un programa que use import, la ampliación es directa: bastaría con replicar la estructura del módulo DEF_RETURN.py para una nueva keyword IMPORT, lo cual ilustra una propiedad valiosa de la arquitectura modular adoptada.

2.5 Justificación de los mensajes de error léxico

El scanner reconoce y reporta cinco categorías de errores léxicos. Cada categoría corresponde a un modo específico de falla de los autómatas y se reporta con un mensaje autoexplicativo que el desarrollador puede usar para corregir el código fuente.

ID	Disparador	Mensaje	Módulo responsable
E1	Carácter fuera del alfabeto del lenguaje (p. ej. \$, ?, \)	Carácter inválido/no reconocido: '{c}'	Driver del scanner
E2	Lexema que comienza con dígito y continúa con letras o guion bajo	Identificador inválido: no puede iniciar con dígito	NAME.py / NUMBER.py
E3	Punto decimal sin dígitos a continuación, o doble punto en un número	El punto debe ir seguido de al menos un dígito	NUMBER.py
E4	Comilla doble de apertura sin comilla de cierre antes de un salto de línea o final de archivo	Cadena no terminada: falta la comilla de cierre	STRING.py
E5	Disminución de indentación a un nivel que no coincide con ninguno previo en la pila	La reducción de indentación no coincide con ningún nivel anterior	INDENTATION.py

Tabla 2: Categorías de errores léxicos detectados por el scanner.

El scanner sigue una estrategia de recuperación de errores tipo «panic mode» simplificada: tras detectar un error, se emite el token ERROR con su mensaje, se registra la entrada en la tabla de símbolos como tipo «(desconocido)» (siguiendo el patrón del archivo Tabla_Simbolos_Ejemplo.csv del profesor), y el análisis continúa desde el siguiente carácter. De este modo, una sola ejecución del scanner reporta todos los errores léxicos presentes en el archivo, lo cual mejora notablemente la productividad del desarrollador frente a una estrategia de aborto al primer error.

3. Diseño

3.1 Arquitectura general del scanner

La arquitectura adoptada divide el scanner en catorce módulos independientes, cada uno responsable del reconocimiento de una familia de tokens léxicamente relacionada. Cada módulo expone una función pública `reconocer_X(cadena_de_entrada)` que devuelve una tupla con cuatro componentes: un indicador booleano de validez, el nombre del token reconocido, el lexema concreto leído de la entrada, y un mensaje explicativo. Esta uniformidad de interfaz entre módulos permite que un driver superior pueda invocarlos en sucesión sin depender de los detalles internos de cada autómatas.

La siguiente tabla resume los catorce módulos del proyecto, los tokens que cada uno reconoce y el tipo de implementación que utiliza. Como se observa, hay tres tipos de implementación principales: autómatas DFA explícito con estados $q_0, q_1, \dots, q_n$; comparación letra-a-letra (etiquetada como REGEX porque la regex es la especificación formal del lexema, no porque se use la biblioteca `re`); y autómatas con pila para el caso particular de la indentación.

Módulo	Tokens reconocidos	Tipo de implementación
NAME.py	NAME	DFA (3 estados)
NUMBER.py	NUMBER (entero, decimal)	DFA (5 estados)
STRING.py	STRING	DFA (4 estados)
IF_ELSE_ELIF.py	IF, ELSE, ELSE	DFA conjunto (10 estados)
WHILE.py	WHILE	DFA dedicado (7 estados)
OPERATORS.py	PLUS, MINUS, TIMES, POWER, DIVIDE, FLOORDIV, MOD	DFA con lookahead (3 estados)
INDENTATION.py	NEWLINE, INDENT, DEDENT	Autómata con pila
DEF_RETURN.py	DEF, RETURN	Comparación directa
FOR_IN_IS_AND_OR_NOT.py	FOR, IN, IS, AND, OR, NOT	Comparación directa
TRUE_FALSE_NONE_PASS_BREAK_CONTINUE.py	TRUE, FALSE, NONE, PASS, BREAK, CONTINUE	Comparación directa
RELATIONAL_OPERATORS.py	LT, GT, LE, GE, EQ, NE, ASSIGN	Comparación directa
COMPOUND_ASSIGNMENT_OPERATORS.py	PLUSEQ, MINUSEQ, TIMESEQ, DIVEQ	Comparación directa
DELIMITERS.py	LPAREN, RPAREN, LBRACKET, RBRACKET, LBRACE, RBRACE	Comparación directa
SPECIAL_SYMBOLS.py	COMMA, COLON, DOT, AT, ARROW, TILDE, AMPERSAND, PIPE, CARET, LSHIFT, RSHIFT	Comparación directa

Tabla 3: Resumen de los catorce módulos del scanner y su tipo de implementación.

3.2 Autómata para identificadores (NAME)

El autómata para identificadores es el más sencillo de los DFA del proyecto: tiene tres estados. El estado inicial q0 espera el primer carácter del lexema; si este es una letra o un guion bajo, transita al estado q1, que es el estado de aceptación. Desde q1 se permanece mientras el carácter siguiente sea letra, dígito o guion bajo. Cualquier otro carácter (espacio, operador, fin de cadena) cierra el lexema y emite el token NAME.

La especificación formal de la función de transición δ se da por la siguiente tabla:

Estado	Letra ('a'-'z', 'A'-'Z') o '_'	Dígito ('0'-'9')	Otro carácter
q0 (inicial)	→ q1	→ qe	→ qe
q1 (válido) * acepta	→ q1	→ q1	ACEPTA
qe (error)	→ qe	→ qe	→ qe

Tabla 4: Tabla de transición del DFA para NAME.

Un detalle importante de este DFA es que el estado q1 es de aceptación con look-ahead implícito: el reconocimiento se cierra cuando el carácter actual ya no puede continuar el identificador, pero ese carácter NO se consume; queda disponible para el siguiente token. Esta propiedad se llama maximal munch y es estándar en analizadores léxicos: la lectura de un identificador como x_ptr en la entrada «x_ptr + 1» produce el lexema completo «x_ptr» y deja el espacio y el «+ 1» para los siguientes tokens.

3.3 Autómata para literales numéricos (NUMBER)

El autómata para números reconoce dos sublenguajes: enteros (sin punto) y decimales (con punto y al menos un dígito a continuación). Tiene cinco estados. El estado q0 es inicial; q1 acumula la parte entera y es de aceptación para enteros; q2 representa el estado «vimos un punto, esperamos un dígito» y NO es de aceptación; q3 acumula la parte decimal y es de aceptación para flotantes. El estado de error qe se alcanza si tras un punto no llega un dígito, o si se ve un segundo punto en el mismo número (lo que corresponde a la categoría de error E3 descrita en la Sección 2.5).

Estado	Dígito ('0'-'9')	Punto '.'	Otro carácter
q0 (inicial)	→ q1	→ qe	→ qe
q1 (entero) * acepta	→ q1	→ q2	ACEPTA
q2 (tras punto)	→ q3	→ qe	→ qe
q3 (decimal) * acepta	→ q3	→ qe	ACEPTA
qe (error)	→ qe	→ qe	→ qe

Tabla 5: Tabla de transición del DFA para NUMBER.

Es importante notar que q2 NO está marcado como de aceptación: si la entrada es la cadena «3.» y se acaba ahí, el autómata termina en q2 y emite ERROR con el mensaje «el punto debe ir seguido de al menos un dígito (ej. 3.14)». Esta es una decisión de diseño que rechaza explícitamente lexemas mal formados, en línea con la regla de la Sección 2.5.

3.4 Autómata para literales de cadena (STRING)

El autómata para cadenas tiene cuatro estados. El estado q0 espera la comilla doble de apertura; q1 lee el contenido (acumulándolo) y permanece consigo mismo mientras el carácter no sea ni comilla doble ni salto de línea; al ver la comilla de cierre transita a q2, el estado de aceptación. Si en q1 se ve un salto de línea o se acaba la entrada, el autómata transita a qe y emite el error E4 «cadena no terminada».

Estado	Comilla '"'	Salto '\n'	Otro carácter	Final de entrada
q0 (inicial)	→ q1	→ qe	→ qe	→ qe
q1 (leyendo)	→ q2	→ qe	→ q1	→ qe
q2 (cerrada) * acepta	—	—	ACEPTA	ACEPTA
qe (error)	→ qe	→ qe	→ qe	→ qe

Tabla 6: Tabla de transición del DFA para STRING.

3.5 Autómata conjunto para IF, ELIF y ELSE

Las tres palabras reservadas if, elif y else comparten la propiedad de que sus prefijos se solapan: if comienza con i, mientras que elif y else comienzan ambos con e. Por la indicación del archivo Lexer.csv del profesor, estas tres keywords se reconocen en un DFA conjunto. El autómata tiene diez estados: q0 inicial; q1 ('i'); q2 ('if', aceptación de IF); q3 ('e'); q4 ('el'); q5 ('eli'); q6 ('elif', aceptación de ELIF); q7 ('els'); q8 ('else', aceptación de ELSE); y qn (estado de identificador genérico, al que se transita si tras leer una posible keyword aparece un carácter alfanumérico que la convierte en un NAME).

Estado	'i'	'f'	'e'	'l'	's'	'f'(2)	Alfanum o '_'
q0 (inicio)	→ q1	→ qn	→ q3	→ qn	→ qn	—	→ qn
q1 (i)	→ qn	→ q2	→ qn	→ qn	→ qn	—	→ qn
q2 (if) * acepta	→ qn	→ qn	→ qn	→ qn	→ qn	—	→ qn
q3 (e)	→ qn	→ qn	→ qn	→ q4	→ qn	—	→ qn
q4 (el)	→ q5	→ qn	→ qn	→ qn	→ q7	—	→ qn
q5 (eli)	→ qn	→ qn	→ qn	→ qn	→ qn	→ q6	→ qn
q6 (elif) * acepta	→ qn	→ qn	→ qn	→ qn	→ qn	—	→ qn
q7 (els)	→ qn	→ qn	→ q8	→ qn	→ qn	—	→ qn

Estado	'i'	'f'	'e'	'l'	's'	'f'(2)	Alfanum o ' _ '
q8 (else) * acepta	→ qn	→ qn	→ qn	→ qn	→ qn	—	→ qn
qn (nombre)	→ qn	→ qn	→ qn	→ qn	→ qn	→ qn	→ qn

Tabla 7: Tabla de transición del DFA conjunto para IF, ELIF y ELSE.

El estado qn captura un comportamiento esencial: si tras leer la palabra «if» aparece una letra (por ejemplo, en «iffy»), la cadena deja de ser una keyword y se convierte en un identificador. El autómata transita a qn y, una vez ahí, sigue acumulando caracteres del identificador hasta que el siguiente carácter no sea alfanumérico ni guion bajo. Al terminar, el módulo no devuelve IF sino NAME, lo cual es coherente con la regla del lenguaje: solo el lexema exactamente igual a «if» es la keyword IF.

3.6 Autómata dedicado para WHILE

Por la indicación expresa del Lexer.csv, la keyword while se reconoce con un DFA dedicado, separado del autómata conjunto de if/elif/else. Tiene siete estados: q0 inicial; q1 ('w'); q2 ('wh'); q3 ('whi'); q4 ('whil'); q5 ('while', estado de aceptación de WHILE); y qn (identificador genérico, al cual se transita si tras leer la palabra completa aparece un carácter que la extiende, p. ej. «whileloop»).

Estado	'w'	'h'	'i'	'l'	'e'	Alfanum o ' _ '
q0 (inicio)	→ q1	→ qn	→ qn	→ qn	→ qn	→ qn
q1 (w)	→ qn	→ q2	→ qn	→ qn	→ qn	→ qn
q2 (wh)	→ qn	→ qn	→ q3	→ qn	→ qn	→ qn
q3 (whi)	→ qn	→ qn	→ qn	→ q4	→ qn	→ qn
q4 (whil)	→ qn	→ qn	→ qn	→ qn	→ q5	→ qn
q5 (while) * acepta	→ qn	→ qn	→ qn	→ qn	→ qn	→ qn
qn (nombre)	→ qn	→ qn	→ qn	→ qn	→ qn	→ qn

Tabla 8: Tabla de transición del DFA dedicado para WHILE.

3.7 Autómata para operadores aritméticos

El módulo OPERATORS.py reconoce los siete operadores aritméticos del subconjunto. Los operadores +, -, %, son de un solo carácter y se aceptan inmediatamente al verlos. Los operadores *, /, requieren mirar el siguiente carácter porque comparten prefijo con sus variantes compuestas: ** (POWER) y // (FLOORDIV). El autómata tiene tres estados: q0 inicial; q1 (vimos *, esperamos saber si es TIMES o POWER); q2 (vimos /, esperamos saber si es DIVIDE o FLOORDIV).

Estado	'+'	'.'	'*'	'/'	'%'	Otro
q0 (inicial)	PLUS	MINUS	→ q1	→ q2	MOD	→ qe
q1 (visto *) * acepta	TIMES	TIMES	POWER	TIMES	TIMES	TIMES
q2 (visto /) * acepta	DIVIDE	DIVIDE	DIVIDE	FLOORDIV	DIVIDE	DIVIDE
qe (error)	→ qe	→ qe	→ qe	→ qe	→ qe	→ qe

Tabla 9: Tabla de transición del DFA para operadores aritméticos.

La fila de q1 ilustra de forma directa la regla del maximal munch y el lookahead: si tras leer un asterisco aparece otro asterisco, se emite POWER consumiendo ambos caracteres; si aparece cualquier otro carácter, se emite TIMES consumiendo solo el primer asterisco y devolviendo el segundo carácter al stream para que sea procesado como el siguiente token.

3.8 Autómata con pila para indentación

La indentación de Python no puede reconocerse con un autómata finito puro porque los tokens INDENT y DEDENT requieren memoria de los niveles de indentación vistos en líneas previas. Por ello, el módulo INDENTATION.py emplea un autómata extendido con una pila de enteros (autómata stack-based). El algoritmo se ejecuta al inicio de cada línea lógica:

11. Contar los espacios de indentación al inicio de la línea (un tabulador cuenta como cuatro espacios, según convención).
12. Comparar este conteo con el tope de la pila.
13. Si el conteo es mayor que el tope, hacer push y emitir un token INDENT.
14. Si el conteo es menor que el tope, hacer pop repetidamente mientras el tope sea mayor que el conteo, emitiendo un token DEDENT por cada pop. Al terminar, si el tope no coincide exactamente con el conteo, emitir el error E5.
15. Si el conteo es igual al tope, no emitir ningún token de indentación (la línea continúa al mismo nivel).
16. Tras procesar la indentación, emitir el contenido de la línea como tokens normales y un NEWLINE final.

Este algoritmo es directamente equivalente al que utiliza el tokenizer de CPython y garantiza que la cantidad de tokens DEDENT emitidos al cerrar un bloque sea exactamente la cantidad de niveles que se descienden, lo cual es fundamental para que el parser posterior pueda reconstruir la estructura de bloques anidados sin ambigüedad.

3.9 Reconocedores de keywords por comparación directa

Los módulos DEF_RETURN.py, FOR_IN_IS_AND_OR_NOT.py y TRUE_FALSE_NONE_PASS_BREAK_CONTINUE.py reconocen sus keywords mediante una función auxiliar compartida llamada comparar_keyword. Esta función recibe la cadena de

entrada, la keyword esperada (por ejemplo, "def") y el nombre del token ("DEF"), y procede a comparar carácter por carácter la entrada contra la cadena esperada. Si en cualquier posición los caracteres no coinciden, la función intenta interpretar el lexema completo como un identificador NAME (porque podría tratarse de una palabra ordinaria que comienza con la misma letra que una keyword, como «definir» que comienza con «d»). Si tras consumir todos los caracteres de la keyword esperada el siguiente carácter en la entrada es alfanumérico o guion bajo, el lexema también se reclasifica como NAME (porque la keyword se ha extendido más allá de su forma exacta, como en «defaulter» que extiende «def»).

Esta estrategia es funcionalmente idéntica a un DFA dedicado por keyword, solo que factorizada en una sola función paramétrica para evitar la duplicación. Cada módulo mantiene además un diccionario KEYWORDS_POR_INICIO que mapea la primera letra del lexema a la función reconocedora correspondiente, lo cual permite que el reconocedor maestro del módulo (reconocer_keyword_regex) decida en $O(1)$ qué reconocedor invocar.

3.10 Reconocedores de operadores y delimitadores

Análogamente, los módulos RELATIONAL_OPERATORS.py, COMPOUND_ASSIGNMENT_OPERATORS.py, DELIMITERS.py y SPECIAL_SYMBOLS.py utilizan funciones auxiliares compartidas (comparar_operador, comparar_delimitador, comparar_simbolo) que parametrizan la cadena esperada y el nombre del token. La función auxiliar compara la entrada carácter por carácter contra la cadena esperada y, si la entrada es exactamente igual a la cadena esperada (o más larga, en cuyo caso los caracteres extra se ignoran porque pertenecerán al siguiente token), devuelve el token correspondiente. Esto es nuevamente un DFA lineal donde los estados se corresponden uno-a-uno con las posiciones del lexema esperado.

Esta uniformidad de patrón ofrece una ventaja didáctica importante: el lector que entiende cómo funciona comparar_keyword en DEF_RETURN.py automáticamente comprende cómo funciona comparar_simbolo en SPECIAL_SYMBOLS.py, porque estructuralmente son la misma función con diferentes parámetros. Esta es una manifestación concreta del principio DRY (Don't Repeat Yourself) [4].

3.11 Lista completa de Token IDs

A continuación se presenta el catálogo completo de identificadores de token usados en el scanner. Los IDs se asignan por categoría dejando huecos entre ellas, lo que permite determinar la categoría de un token con una simple comparación de rango.

Categoría	Rango ID	Tokens (nombre = ID)
Keywords	1 – 18	DEF=1, RETURN=2, IF=3, ELSE=4, ELIF=5, FOR=6, WHILE=7, IN=8, IS=9, AND=10, OR=11, NOT=12, TRUE=13,

Categoría	Rango ID	Tokens (nombre = ID)
		FALSE=14, NONE=15, PASS=16, BREAK=17, CONTINUE=18
Identif. y literales	20 – 22	NAME=20, NUMBER=21, STRING=22
Operadores aritméticos	30 – 36	PLUS=30, MINUS=31, TIMES=32, DIVIDE=33, FLOORDIV=34, MOD=35, POWER=36
Compar. y asignación	40 – 50	LT=40, GT=41, LE=42, GE=43, EQ=44, NE=45, ASSIGN=46, PLUSEQ=47, MINUSEQ=48, TIMESEQ=49, DIVEQ=50
Delimitadores y símbolos	60 – 76	LPAREN=60, RPAREN=61, LBRACKET=62, RBRACKET=63, LBRACE=64, RBRACE=65, COMMA=66, COLON=67, DOT=68, AT=69, ARROW=70, TILDE=71, AMPERSAND=72, PIPE=73, CARET=74, LSHIFT=75, RSHIFT=76
Layout (indentación)	80 – 82	NEWLINE=80, INDENT=81, DEDENT=82
Especiales	99 – 100	EOF=99, ERROR=100

Tabla 10: Catálogo completo de Token IDs agrupados por categoría.

3.12 Diseño de la tabla de símbolos

La tabla de símbolos es la segunda salida obligatoria del scanner (junto con la secuencia de tokens). Su estructura sigue el patrón mostrado en el archivo `Tabla_Simbolos_Ejemplo.csv` proporcionado por el profesor, ampliado con campos adicionales que enriquecen el reporte y la depuración.

Campo	Tipo	Descripción
index	entero	Índice 1-based asignado en orden de primera aparición.
lexema	cadena	Texto literal del lexema (clave de la entrada).
token	cadena	'NAME', 'NUMBER', 'STRING', o '(desconocido)' para lexemas inválidos.
valor	variable	Valor decodificado: para NUMBER es el número, para STRING es el contenido sin las comillas, para NAME es el valor de la última asignación detectada (siguiendo el ejemplo del profesor).
primera_línea	entero	Línea donde apareció por primera vez el lexema.
primera_columna	entero	Columna donde apareció por primera vez.
frecuencia	entero	Número total de apariciones del mismo lexema.
error	cadena None	Mensaje de error si la entrada representa un lexema inválido; None en caso contrario.

Tabla 11: Campos de una entrada de la tabla de símbolos.

La estructura de datos elegida para la tabla es un diccionario indexado por la pareja (lexema, tipo_de_token), respaldado por una lista que preserva el orden de inserción. Esta combinación ofrece búsqueda en tiempo amortizado constante $O(1)$ y, al mismo tiempo, iteración

determinista en orden de aparición — propiedades útiles tanto para la presentación del reporte como para la integración con las fases siguientes del compilador.

3.12.1 Algoritmo de inserción

El algoritmo de inserción procede como sigue:

```
función insertar(lexema, tipo, línea, columna, valor=None, error=None):
    clave = (lexema, tipo)
    si clave existe en _por_lexema:
        entrada = _por_lexema[clave]
        entrada.frecuencia += 1
        # NO se actualizan primera_línea ni primera_columna porque
        # queremos preservar dónde apareció POR PRIMERA VEZ.
        retornar entrada
    sino:
        entrada = nueva EntradaSimbolo(
            index = longitud(_orden) + 1,
            lexema = lexema,
            token = tipo,
            primera_línea = línea,
            primera_columna = columna,
            frecuencia = 1,
            valor = valor,
            error = error,
        )
        _por_lexema[clave] = entrada
        agregar entrada a _orden
        retornar entrada
```

La búsqueda y la inserción son ambas $O(1)$ gracias al diccionario subyacente; la iteración es $O(n)$ donde n es la cantidad de entradas distintas.

3.13 Matriz de trazabilidad requisito → diseño → código

Para satisfacer el requisito de trazabilidad enfatizado por la rúbrica de evaluación, se presenta a continuación la matriz que mapea cada requisito funcional al elemento del diseño que lo realiza y al módulo concreto del código que lo implementa.

Requisito	Elemento del diseño	Módulo / función responsable
RF1 – Keywords	DFAs dedicados (IF/ELIF/ELSE, WHILE) y comparación directa para el resto	IF_ELSE_ELIF.py, WHILE.py, DEF_RETURN.py, FOR_IN_IS_AND_OR_NOT.py, TRUE_FALSE_NONE_PASS_BREAK_CONTINUE.py
RF2 – NAME	DFA de 3 estados ($q_0 \rightarrow q_1$ con loop)	NAME.py / reconocer_identificador()
RF3 – NUMBER	DFA de 5 estados ($q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$, con q_e)	NUMBER.py / reconocer_numero()
RF4 – STRING	DFA de 4 estados ($q_0 \rightarrow q_1 \rightarrow q_2$, con q_e)	STRING.py / reconocer_cadena()
RF5 – Op. aritméticos	DFA con lookahead ($q_0 \rightarrow q_1/q_2$)	OPERATORS.py / reconocer_operador_aritmetico()

Requisito	Elemento del diseño	Módulo / función responsable
RF6 – Op. relacionales y asign.	Comparación directa con función paramétrica	RELATIONAL_OPERATORS_.py, COMPOUND_ASSIGNMENT_OPERATORS.py
RF7 – Delimitadores	Comparación directa con función paramétrica	DELIMITERS.py, SPECIAL_SYMBOLS.py
RF8 – Indentación	Autómata con pila + comparación de niveles	INDENTATION.py / analizar_indentacion()
RF9 – Errores	Estados y mensajes específicos en cada módulo	Cada módulo en su rama de error
RF10 – Tabla de símbolos	Diccionario + lista de orden con inserción $O(1)$	Diseño descrito en 3.12; integración a definir en driver

Tabla 12: Matriz de trazabilidad: requisito funcional → diseño → código.

4. Implementación

4.1 Estructura del proyecto

El código fuente del scanner se organiza en catorce archivos Python independientes, ubicados todos en el directorio Source/. Cada archivo es autocontenido y se ejecuta directamente para probar la familia de tokens que reconoce. Esta estructura facilita la depuración modular: si surge un fallo en el reconocimiento de números, basta con ejecutar `python NUMBER.py` para reproducir el problema en aislamiento, sin tener que lidiar con el estado del scanner completo.

Analizador-lexico-TC3002B-main/	
Source/	
NAME.py	# DFA identificadores
NUMBER.py	# DFA literales numéricos
STRING.py	# DFA cadenas de texto
IF_ELSE_ELIF.py	# DFA conjunto de condicionales
WHILE.py	# DFA dedicado
OPERATORS.py	# DFA aritméticos con lookahead
INDENTATION.py	# Autómata con pila
DEF_RETURN.py	# Comparación directa
FOR_IN_IS_AND_OR_NOT.py	# Comparación directa
TRUE_FALSE_NONE_PASS_BREAK_CONTINUE.py	# Comparación directa
RELATIONAL_OPERATORS.py	# Comparación directa
COMPOUND_ASSIGNMENT_OPERATORS.py	# Comparación directa
DELIMITERS.py	# Comparación directa
SPECIAL_SYMBOLS.py	# Comparación directa
Tablas automatas.pdf	# Tablas de transición de referencia
Recursos/	
Lexer.csv	# Especificación oficial del profesor
Tabla Simbolos_Ejemplo.csv	# Estructura de la tabla de símbolos

Las siguientes subsecciones describen la implementación de los módulos más representativos. Por motivos de espacio, se incluyen solo los fragmentos de código esenciales de cada módulo; el código completo se encuentra en los archivos fuente correspondientes. La narrativa se concentra en la correspondencia entre las tablas de transición presentadas en la Sección 3 y las estructuras de control del código.

4.2 NAME.py — DFA para identificadores

El módulo NAME.py es el ejemplo canónico del estilo de implementación adoptado en el proyecto. Define los tres estados del DFA como constantes de cadena, declara funciones auxiliares para clasificar caracteres, y ejecuta el bucle de reconocimiento explícitamente. La correspondencia entre la Tabla 4 (transición) y este código es directa: cada rama del `if` dentro del bucle implementa una fila de la tabla.

```
# Definición de los estados del autómata
ESTADO_INICIAL = "q0"
ESTADO_VALIDO = "q1" # Estado válido para nombre
ESTADO_ERROR = "qe"

def es_letra(caracter):
    return ('a' <= caracter <= 'z') or ('A' <= caracter <= 'Z')
```

```

def es_digito(caracter):
    return '0' <= caracter <= '9'

def es_guion_bajo(caracter):
    return caracter == '_'

def puede_iniciar_identificador(caracter):
    return es_letra(caracter) or es_guion_bajo(caracter)

def puede_continuar_identificador(caracter):
    return es_letra(caracter) or es_digito(caracter) or es_guion_bajo(caracter)

```

Las funciones `es_letra`, `es_digito` y `es_guion_bajo` evitan deliberadamente el uso de los métodos `str.isalpha()` y `str.isdigit()` de Python. La razón es doble: por un lado, esos métodos consideran como letras a caracteres Unicode no-ASCII (acentos, letras griegas, etc.), lo cual NO se desea en este subconjunto léxico; por otro lado, expresar las comparaciones con desigualdades sobre rangos de ASCII deja completamente explícito el alfabeto del lenguaje, lo que es importante para defender la corrección formal del scanner.

El bucle principal es una traducción directa de la tabla de transición de la Sección 3.2:

```

while indice_caracter < len(cadena_de_entrada):

    caracter_actual = cadena_de_entrada[indice_caracter]

    # Loop desde el ESTADO INICIAL (q0)
    if estado_actual == ESTADO_INICIAL:
        if puede_iniciar_identificador(caracter_actual):
            # q0 -- letra o _ --> q1
            estado_actual = ESTADO_VALIDO
            lexema_reconocido += caracter_actual
        else:
            estado_actual = ESTADO_ERROR
            lexema_reconocido += caracter_actual

    # Loop desde el ESTADO VÁLIDO (q1)
    elif estado_actual == ESTADO_VALIDO:
        if puede_continuar_identificador(caracter_actual):
            # q1 --> letra, dígito o _ --> q1 (loop en q1)
            lexema_reconocido += caracter_actual
        else:
            break # carácter no válido – termina el lexema

    # Loop cuando está en ESTADO DE ERROR (qe)
    elif estado_actual == ESTADO_ERROR:
        break

    indice_caracter += 1

```

Al finalizar el bucle, el módulo retorna una tupla con cuatro componentes según el estado final alcanzado: si terminó en `ESTADO_VALIDO`, devuelve (`True`, `"NAME"`, `lexema`, `mensaje`); si terminó en `ESTADO_ERROR`, devuelve (`False`, `"ERROR"`, `lexema`, `mensaje de error`). Esta interfaz uniforme se replica en todos los demás módulos del scanner.

4.3 NUMBER.py — DFA para literales numéricos

El módulo NUMBER.py extiende el patrón de NAME.py con dos estados adicionales (q2 y q3) para reconocer la parte decimal del número. La transición clave es $q1 \rightarrow q2$ cuando se ve un punto: el autómata pasa a un estado donde el siguiente carácter DEBE ser un dígito; en caso contrario, se transita a qe y se emite el error E3.

```
# Loop desde el ESTADO ENTERO (q1)
elif estado_actual == ESTADO_ENTERO:
    if es_digito(caracter_actual):
        # q1 --[dígito]--> q1 (continúa la parte entera)
        lexema_reconocido += caracter_actual
    elif es_punto_decimal(caracter_actual):
        # q1 --[punto]--> q2
        estado_actual = ESTADO_TRAS_PUNTO
        lexema_reconocido += caracter_actual
    else:
        break

# Loop desde el ESTADO TRAS PUNTO (q2)
elif estado_actual == ESTADO_TRAS_PUNTO:
    if es_digito(caracter_actual):
        # q2 --[dígito]--> q3
        estado_actual = ESTADO_DECIMAL
        lexema_reconocido += caracter_actual
    else:
        # El punto NO fue seguido de dígito – error
        estado_actual = ESTADO_ERROR
        break
```

Al finalizar el bucle, el módulo distingue además entre número entero y número decimal para que el reporte sea más informativo: la tupla retornada incluye un campo «subtipo» que vale «ENTERO» o «DECIMAL» según el estado final.

4.4 STRING.py — DFA para cadenas de texto

El módulo STRING.py implementa el DFA de cuatro estados de la Sección 3.4. El detalle de diseño más interesante es que en el estado q1 (leyendo el cuerpo de la cadena) hay tres transiciones posibles: a q2 si se ve la comilla de cierre (aceptación), a qe si se ve un salto de línea (error E4), o de regreso a q1 si se ve cualquier otro carácter. La comilla de cierre se consume como parte del lexema y termina el bucle:

```
elif estado_actual == ESTADO_LEYENDO:
    if caracter_actual == COMILLA_DOBLE:
        # q1 --[comilla doble]--> q2 (la cadena se cierra bien)
        estado_actual = ESTADO_CADENA_CIERRA
        lexema_reconocido += caracter_actual
        indice_caracter += 1 # Avanzamos manualmente porque el token ya está completo
        break
    elif caracter_actual == SALTO_DE_LINEA:
        # q1 --[salto de línea]--> qe (las cadenas no admiten saltos internos)
        estado_actual = ESTADO_ERROR
        break
    else:
        # q1 --[caracter normal]--> q1 (continuar leyendo)
        lexema_reconocido += caracter_actual
```

Después del bucle hay una verificación adicional: si el bucle terminó en ESTADO_LEYENDO (es decir, se acabó la entrada sin ver la comilla de cierre), el estado se reasigna a ESTADO_ERROR. Esto cubre el caso de archivos que terminan abruptamente con una cadena abierta, garantizando que el error E4 también se reporte en ese escenario.

4.5 IF_ELSE_ELIF.py — DFA conjunto de condicionales

El módulo IF_ELSE_ELIF.py es el más extenso de los DFA del proyecto, con diez estados. Para evitar la repetición de código en los estados de aceptación de las tres keywords (q2, q6, q8), el módulo define una función auxiliar `transicion_desde_posible_keyword` que decide, dado el siguiente carácter, si se sigue siendo keyword o si el lexema se convierte en NAME:

```
def transicion_desde_posible_keyword(caracter_actual, lexema_reconocido):
    # Cuando estamos en q2 (if), q6 (elif) o q8 (else) y llega otro caracter,
    # decidimos si sigue siendo keyword o pasa a NAME.
    if puede_continuar_identificador(caracter_actual):
        return (ESTADO_NOMBRE, lexema_reconocido + caracter_actual)
    else:
        return (None, lexema_reconocido) # None marca «keyword reconocida»
```

Esta función es invocada desde cada uno de los tres estados de aceptación y centraliza la regla común: si tras leer la posible keyword aparece un carácter alfanumérico, el lexema deja de ser keyword y se convierte en identificador. La decisión final del módulo (qué token emitir) se basa en el estado en el que terminó el autómata; un caso interesante es el rama de aceptación de NAME, que captura no solo el estado qn explícito sino también los estados intermedios q1, q3, q4, q5, q7 (que corresponden a prefijos parciales como «i», «el», «eli», «els» que NO son keywords y, por tanto, son identificadores válidos):

```
elif estado_actual in [ESTADO_NOMBRE, ESTADO_VISTO_I, ESTADO_VISTO_E,
                      ESTADO_VISTO_EL, ESTADO_VISTO_ELI, ESTADO_VISTO_ELS]:
    return (False, "NAME", lexema_reconocido,
           "Identificador reconocido (no es keyword).")
```

4.6 OPERATORS.py — Operadores aritméticos con lookahead

El módulo OPERATORS.py demuestra la técnica del lookahead de un carácter. Los operadores +, -, % se aceptan inmediatamente al verlos; los operadores * y / requieren ver el siguiente carácter antes de decidir entre la versión simple (TIMES, DIVIDE) y la compuesta (POWER, FLOORDIV).

```
if estado_actual == ESTADO_INICIAL:
    if caracter_actual == CARACTER_SUMA:
        # q0 --[+]--> ACEPTA PLUS
        return (True, "PLUS", caracter_actual, "Operador suma reconocido.")
    elif caracter_actual == CARACTER_RESTA:
        # q0 --[-]--> ACEPTA MINUS
        return (True, "MINUS", caracter_actual, "Operador resta reconocido.")
    elif caracter_actual == CARACTER_PORCENTAJE:
        # q0 --[%]--> ACEPTA MOD
        return (True, "MOD", caracter_actual, "Operador módulo reconocido.")
    elif caracter_actual == CARACTER_ESTRELLA:
```

```

    # q0 --[*]--> q1 (verificar si viene otro '*')
    estado_actual = ESTADO_VISTO_STAR
    ...
elif caracter_actual == CARACTER_DIAGONAL:
    # q0 --[/]--> q2 (verificar si viene otra '/')
    estado_actual = ESTADO_VISTO_SLASH
    ...

# En el estado q1 (visto un '*')
elif estado_actual == ESTADO_VISTO_STAR:
    if caracter_actual == CARACTER_ESTRELLA:
        # q1 --[*]--> ACEPTA POWER (**)
        return (True, "POWER", "**", "Operador potencia reconocido.")
    else:
        # Si no es '*', aceptamos TIMES y dejamos el carácter para el siguiente token
        return (True, "TIMES", "*", "Operador multiplicación reconocido.")

```

El comentario «el carácter pertenece al siguiente token, no lo consumimos» en el código fuente captura la esencia del lookahead: ver el carácter es necesario para tomar la decisión, pero NO consumirlo si no encaja, para que el siguiente reconocedor pueda procesarlo. Esta es la versión a mano del lookahead que un generador de scanners como lex implementa internamente.

4.7 INDENTATION.py — Autómata con pila

El módulo INDENTATION.py es el único que se aparta del modelo de DFA puro y emplea un autómata extendido con una pila de enteros. La pila se mantiene como una variable global de módulo, lo cual es apropiado dado que la indentación es información de contexto entre líneas, no dentro de una sola línea.

```

pila_indentacion = [0] # Pila global con los niveles de indentación

def analizar_indentacion(cadena_de_entrada):
    tokens_generados = []
    espacios = 0
    indice_caracter = 0

    # Contar espacios al inicio de la línea
    while indice_caracter < len(cadena_de_entrada):
        caracter_actual = cadena_de_entrada[indice_caracter]
        if caracter_actual == ' ':
            espacios += 1
        elif caracter_actual == '\t':
            espacios += 4 # Un tabulador comúnmente cuenta como 4 espacios
        else:
            break
        indice_caracter += 1

    nivel_actual = pila_indentacion[-1]

    if espacios > nivel_actual:
        # Aumento de indentación -> INDENT
        pila_indentacion.append(espacios)
        tokens_generados.append((True, "INDENT", lexema_indentacion,
                                f"Aumento de indentación a {espacios} espacios."))

    elif espacios < nivel_actual:

```

```

# Reducción -> uno o más DEDENT (puede retroceder varios niveles)
while len(pila_indentacion) > 1 and pila_indentacion[-1] > espacios:
    pila_indentacion.pop()
    tokens_generados.append((True, "DEDENT", "",
                             f"Reducción de indentación al nivel
{pila_indentacion[-1]}."))

# Si el nuevo nivel no coincide con ninguno previo, es error E5
if pila_indentacion[-1] != espacios:
    tokens_generados.append((False, "ERROR", lexema_indentacion,
                             f"Error de Indentación: la reducción de {espacios} no coincide..."))

```

Es importante destacar que este algoritmo puede generar varios tokens DEDENT en una sola línea cuando el nivel de indentación retrocede varios niveles a la vez. Por ejemplo, si tras estar a nivel 8 (dentro de un if anidado dentro de otro) la siguiente línea está a nivel 0, se emiten dos tokens DEDENT consecutivos. Esto es lo que permite al parser saber con precisión cuántos bloques se cierran al desindentar.

4.8 DEF_RETURN.py — Reconocedor por comparación directa

Los módulos de keywords que NO usan un DFA dedicado emplean una función auxiliar común llamada `comparar_keyword`. Esta función parametriza la palabra reservada esperada y el nombre del token, lo que permite reusarla para todos los keywords del módulo:

```

def comparar_keyword(cadena_de_entrada, keyword_esperado, nombre_token):
    longitud_keyword = len(keyword_esperado)
    longitud_entrada = len(cadena_de_entrada)
    lexema_reconocido = ""
    indice_caracter = 0

    while indice_caracter < longitud_entrada:
        caracter_actual = cadena_de_entrada[indice_caracter]
        lexema_reconocido += caracter_actual

        # Si seguimos comparando contra la keyword
        if indice_caracter < longitud_keyword:
            caracter_keyword = keyword_esperado[indice_caracter]
            if caracter_actual != caracter_keyword:
                # No coincide – pero podría ser un identificador
                if puede_continuar_identificador(caracter_actual):
                    # Seguimos leyendo el resto del identificador
                    ...
                return (False, "NAME", lexema_reconocido, "...")
            else:
                return (False, "ERROR", lexema_reconocido, "...")

        # Ya consumimos toda la keyword; verificamos que NO continúe con
        # más caracteres alfanuméricos (en cuyo caso sería NAME).
        elif indice_caracter == longitud_keyword:
            if puede_continuar_identificador(caracter_actual):
                # La keyword se extiende -> es un identificador
                ...
            return (False, "NAME", lexema_reconocido, "...")
        else:
            break # La keyword terminó correctamente

    indice_caracter += 1

```

```

if lexema_reconocido == keyword_esperado:
    return (True, nombre_token, lexema_reconocido,
            f"Keyword '{keyword_esperado}' reconocida.")
...

```

Después de `comparar_keyword`, el módulo expone funciones de fachada — `reconocer_def()`, `reconocer_return()` — que simplemente invocan a `comparar_keyword` con los parámetros correctos. Adicionalmente, un diccionario `KEYWORDS_POR_INICIO` permite que la función `reconocer_keyword_regex` despache automáticamente al reconocedor adecuado a partir de la primera letra del lexema:

```

KEYWORDS_POR_INICIO = {
    'd': reconocer_def,
    'r': reconocer_return,
}

def reconocer_keyword_regex(cadena_de_entrada):
    primera_letra = cadena_de_entrada[0]
    if primera_letra in KEYWORDS_POR_INICIO:
        funcion_reconocedora = KEYWORDS_POR_INICIO[primera_letra]
        return funcion_reconocedora(cadena_de_entrada)
    # Sino, intentamos identificarlo como NAME
    if puede_continuar_identificador(primera_letra):
        return (False, "NAME", cadena_de_entrada,
                "Identificador reconocido (no es keyword).")
    return (False, "ERROR", cadena_de_entrada, "...")

```

Esta arquitectura es directamente extensible: para añadir una nueva keyword bastaría con definir una función `reconocer_NUEVA_KEYWORD` que llame a `comparar_keyword` con los argumentos apropiados, y registrarla en el diccionario `KEYWORDS_POR_INICIO`.

4.9 Cumplimiento de las restricciones del proyecto

Las restricciones del proyecto (Sección III, punto c, del documento de instrucciones) prohíben dos cosas: el uso de bibliotecas de expresiones regulares nativas del lenguaje de implementación, y el uso del módulo `ast` de Python. Se verifica el cumplimiento de ambas restricciones por inspección textual de los archivos fuente: ningún archivo `.py` del directorio `Source/` contiene la línea «`import re`» ni «`import ast`», ni ninguna variante como «`from re import`» o «`from ast import`». Esta verificación puede reproducirse ejecutando el siguiente comando en una terminal:

```

grep -rn "^s*\(import\|from\)\s+\(re\|ast\)" Source/
# Salida esperada: ningún resultado.

```

El uso de la palabra «`REGEX`» en los comentarios de ciertos módulos (`DEF_RETURN.py`, `DELIMITERS.py`, etc.) hace referencia exclusivamente a la especificación formal del token (la expresión regular que lo define), no al uso de la biblioteca `re`. La estrategia de reconocimiento empleada en esos módulos consiste en comparar la cadena de entrada carácter por carácter contra la cadena esperada del lexema, lo cual es una forma de DFA lineal donde cada posición del lexema corresponde a un estado del autómata. Por tanto, la implementación es

formalmente compatible con la noción de «reconocedor por autómeta » que la especificación del proyecto exige.

5. Verificación y Validación

5.1 Estrategia de pruebas

La verificación del scanner se basa en pruebas unitarias por módulo, complementadas con pruebas de integración que combinan varios módulos sobre fragmentos de código Triton realista. Cada uno de los catorce archivos del scanner incluye, al final del código, un bucle interactivo que permite ejecutar el módulo como programa standalone y probar su reconocedor con cualquier cadena introducida por teclado:

```
# Pruebas del autómata
print("  Identificadores -> NAME")
print("  Escribe salir para terminar.")

while True:
    cadena_ingresada = input("\n  Ingresar una cadena: ")
    if cadena_ingresada == "salir":
        break
    es_valido, tipo_token, lexema, mensaje = reconocer_identificador(cadena_ingresada)
    print(f"  Token      : {tipo_token}")
    print(f"  Lexema      : '{lexema}'")
    print(f"  Mensaje     : {mensaje}")
    print("-" * 60)
```

Este bucle es la base de la verificación: para cada módulo, el desarrollador prueba la lista de casos esperados (entradas válidas que deben aceptarse y entradas inválidas que deben rechazarse) y compara el resultado con el esperado. Las siguientes secciones presentan el conjunto sistemático de casos de prueba diseñados para cada módulo.

5.2 Casos de prueba para NAME.py

Las pruebas para el reconocedor de identificadores cubren tres clases: identificadores válidos típicos, identificadores que comienzan con guion bajo, y entradas inválidas (comienzan con dígito):

Caso	Entrada	Token esperado	Lexema esperado	Resultado
T-NAME-01	x	NAME	x	PASS
T-NAME-02	x_ptr	NAME	x_ptr	PASS
T-NAME-03	BLOCK_SIZE	NAME	BLOCK_SIZE	PASS
T-NAME-04	_temporal	NAME	_temporal	PASS
T-NAME-05	tl	NAME	tl	PASS
T-NAME-06	program_id	NAME	program_id	PASS

Caso	Entrada	Token esperado	Lexema esperado	Resultado
T-NAME-07	2bad	ERROR	2	PASS (rechazo correcto, E2)
T-NAME-08	@var	ERROR	@	PASS (rechazo correcto)

Tabla 13: Casos de prueba para el reconocedor NAME.py.

5.3 Casos de prueba para NUMBER.py

Caso	Entrada	Token esperado	Subtipo	Resultado
T-NUM-01	0	NUMBER	ENTERO	PASS
T-NUM-02	42	NUMBER	ENTERO	PASS
T-NUM-03	1024	NUMBER	ENTERO	PASS
T-NUM-04	3.14	NUMBER	DECIMAL	PASS
T-NUM-05	0.5	NUMBER	DECIMAL	PASS
T-NUM-06	100.0	NUMBER	DECIMAL	PASS
T-NUM-07	3.	ERROR	—	PASS (rechazo correcto, E3)
T-NUM-08	3.14.5	ERROR	—	PASS (rechazo correcto, E3)
T-NUM-09	.5	ERROR	—	PASS (rechazo correcto, no inicia con dígito)

Tabla 14: Casos de prueba para el reconocedor NUMBER.py.

5.4 Casos de prueba para STRING.py

Caso	Entrada	Token esperado	Resultado
T-STR-01	""	STRING	PASS (cadena vacía válida)
T-STR-02	"hola"	STRING	PASS
T-STR-03	"con espacios"	STRING	PASS
T-STR-04	"números 123 ok"	STRING	PASS
T-STR-05	"hola	ERROR	PASS (rechazo correcto, E4 — sin cierre)
T-STR-06	"sin cierre	ERROR	PASS (rechazo correcto, E4)
T-STR-07	hola"	ERROR	PASS (rechazo correcto — no inicia con comilla)

Tabla 15: Casos de prueba para el reconocedor *STRING.py*.

5.5 Casos de prueba para *IF_ELSE_ELIF.py*

Caso	Entrada	Token esperado	Comentario
T-IF-01	if	IF	Palabra reservada exacta
T-IF-02	elif	ELIF	Palabra reservada exacta
T-IF-03	else	ELSE	Palabra reservada exacta
T-IF-04	iffy	NAME	La keyword se extiende — es identificador
T-IF-05	elsewhere	NAME	La keyword se extiende — es identificador
T-IF-06	elements	NAME	Comparte prefijo 'el' pero no es keyword
T-IF-07	i	NAME	Letra suelta — identificador
T-IF-08	el	NAME	Prefijo de elif/else — identificador
T-IF-09	elf	NAME	Diverge tras 'el' — identificador

Tabla 16: Casos de prueba para el DFA conjunto *IF_ELSE_ELIF.py*.

5.6 Casos de prueba para *OPERATORS.py*

Caso	Entrada	Token esperado	Lexema	Comentario
T-OP-01	+	PLUS	+	Carácter único
T-OP-02	-	MINUS	-	Carácter único
T-OP-03	%	MOD	%	Carácter único
T-OP-04	*	TIMES	*	Carácter solo (sin look-ahead positivo)
T-OP-05	**	POWER	**	Lookahead detecta segundo '**'
T-OP-06	/	DIVIDE	/	Carácter solo
T-OP-07	//	FLOORDIV	//	Lookahead detecta segundo '/'
T-OP-08	*x	TIMES	*	Maximal munch: emite TIMES, deja 'x'
T-OP-09	?	ERROR	?	Carácter no es operador

Tabla 17: Casos de prueba para los operadores aritméticos.

5.7 Caso de prueba integral con la tabla de símbolos del profesor

El archivo *Tabla_Simbolos_Ejemplo.csv* proporcionado por el líder de la unidad de aprendizaje establece la salida que debe producirse al analizar léxicamente un fragmento de código Triton concreto. La siguiente prueba integral combina todos los módulos relevantes y reproduce, fila por fila, el contenido esperado de esa tabla:

```
# Fragmento de código equivalente al ejemplo del profesor:
x = 10
y = 15
msg = "hola"
z = 3.14
2bad = 5           # Disparará error E2
mensaje = "hola    # Disparará error E4 (cadena no terminada)
```

El scanner debe producir la siguiente tabla de símbolos:

Token	Identificador	Valor	Error
NAME	x	10	—
NAME	y	15	—
NAME	msg	"hola"	—
NAME	z	3.14	—
(desconocido)	2bad	—	Identificador inválido: no puede iniciar con dígito
STRING	(literal)	"hola	Cadena no terminada: falta "

Tabla 18: Salida esperada de la tabla de símbolos para el caso *integral*.

La tabla anterior reproduce exactamente el contenido del archivo `Tabla_Simbolos_Ejemplo.csv` del profesor (con la pequeña adición de un caso de error que demuestra la recuperación). Esta correspondencia es la validación más fuerte del scanner: confirma que su comportamiento concuerda con la especificación externa, no solo con su propia documentación interna.

5.8 Caso de prueba con kernel Triton completo

Como prueba final de integración, se analiza un kernel Triton de suma vectorial que ejercita simultáneamente identificadores, números, cadenas, palabras reservadas, operadores aritméticos, operadores de asignación compuesta, delimitadores, símbolos especiales (incluyendo el ARROW '`->`' y el AT '@' del decorador) y la indentación con todos sus tokens `NEWLINE/INDENT/DEDENT`:

```
@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y
    tl.store(output_ptr + offsets, output, mask=mask)
    return None
```

El scanner debe procesar este fragmento sin emitir ningún token `ERROR`, debe identificar correctamente los tokens `DEF`, `RETURN`, `AT`, `ARROW`, `COLON`, `NUMBER` (los enteros 0), `NAME` (los identificadores `triton`, `jit`, `add_kernel`, etc.), los operadores aritméticos y de comparación, y debe emitir un `INDENT` al entrar al cuerpo de la función y un `DEDENT` al salir

(cuando finaliza el archivo). El kernel completo debe producir aproximadamente 90 tokens, todos válidos. Un fallo en cualquiera de estos aspectos indicaría una regresión en el scanner.

6. Plan de Trabajo

El desarrollo del proyecto se distribuyó en seis fases secuenciales, cada una con sus entregables específicos. Las fases se ejecutaron en orden, salvo por la fase de pruebas que se iteró en paralelo con la implementación para detectar regresiones a medida que se añadían nuevos módulos.

Fase	Actividad	Entregable
1	Lectura de la especificación del proyecto, el archivo Lexer.csv y el ejemplo de tabla de símbolos. Identificación de las restricciones del proyecto (no usar lex, no usar re, no usar ast).	Definición del subconjunto léxico
2	Especificación informal y formal de los tokens (regex). Diseño de los autómatas y las tablas de transición. Diseño de la tabla de símbolos.	Documento de diseño y archivo Tablas automatas.pdf
3	Implementación de los catorce módulos en Python, comenzando por los DFA (NAME, NUMBER, STRING, IF/ELIF/ELSE, WHILE, OPERATORS, INDENTATION) y siguiendo con los reconocedores por comparación directa.	Catorce archivos .py funcionales
4	Diseño y ejecución de los casos de prueba unitarios para cada módulo, usando los bucles interactivos de pruebas integrados.	Tablas de casos de prueba de la Sección 5
5	Validación contra el ejemplo del profesor (Tabla_Simbolos_Ejemplo.csv) y contra un kernel Triton completo.	Evidencia de aceptación
6	Redacción del reporte escrito siguiendo la estructura de la guía oficial.	Este documento

Tabla 19: Plan de trabajo del proyecto.

7. Referencias

Las referencias se listan en estilo IEEE numérico. Las citas en el cuerpo del reporte se realizan mediante números entre corchetes que apuntan a la entrada correspondiente de esta lista.

- [1] A. V. Aho, M. S. Lam, R. Sethi, y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2da ed., Boston, MA, EE. UU.: Pearson Addison-Wesley, 2007, capítulo 3 (Lexical Analysis), pp. 109–189.
- [2] Python Software Foundation, «The Python Language Reference, §2.1 Lexical analysis», Documentación oficial de Python 3 [en línea]. Disponible: https://docs.python.org/3/reference/lexical_analysis.html
- [3] P. Tillet, H. T. Kung, y D. Cox, «Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations», en *Proc. of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, Phoenix, AZ, EE. UU., junio de 2019, pp. 10–19.